自分の実験を説明するためのガイド

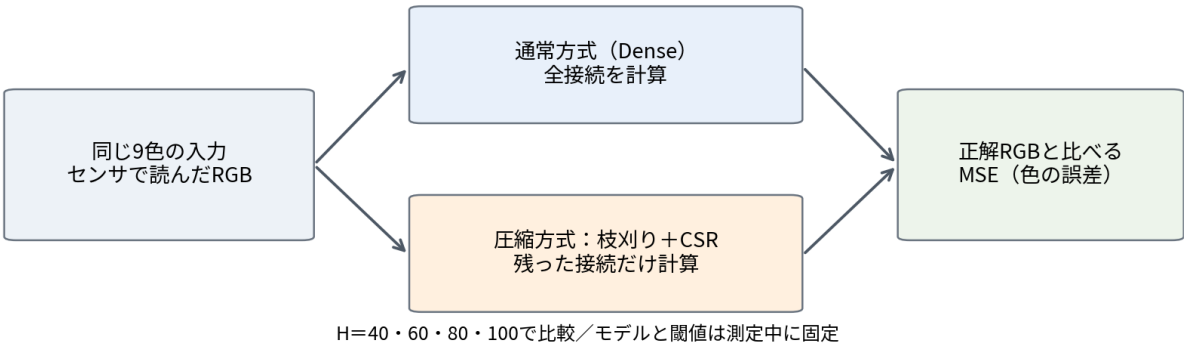
式・コード・測定・結果のつながりを理解する

2026年11月13日

1．何を調べた実験なのか

テーマは「色を補正するモデルを小さく・速くしても、色の正確さを保てるか」である。 このPDFは、提出用レポートの内容を理解するための解説資料として読む。

紙をセンサで読むと、赤・緑・青の3つの値（RGB）が出る。その値を、元の色に近づける計算が色補正である。学習済みの数値と計算の仕組みをモデルと呼ぶ。



通常方式をDense、小さい重みを0にする枝刈りと、0を省いて保存するCSRを組み合わせた方式を圧縮方式と呼んだ。同じ読取データを両方式へ渡して比べた。

数字	自分の実験では何を意味するか
9色	3×3の紙にある色の数
135行	学習に使った入力と正解の組数。同じ色の繰返しを含む
H＝40・60・80・100	隠れ層それぞれの計算単位の数。色数や測定回数とは別
8条件	4種類のH×Dense／枝刈り＋CSR
10回	9色を最後まで読み取った回数
80個のMSE	1回から8条件の結果が出るため、8×10個

PCの役割は学習・枝刈り・CSRの準備である。**Arduinoの役割**は完成した重みを使って色を補正すること。学習した数値は.hファイルに保存した。

結論は、**容量と計算時間は減ったが、圧縮後の色の誤差は増えた**というもの。この理由を、次のページから順に読む。

2．センサの値をどうやって色に変えたか

黒と白を登録した理由

黒と白を基準にして、各RGB成分の入力をそろえた。これを校正と正規化という。

$$\text{入力} = \frac{\text{読取値} - \text{黒の値}}{\text{白の値} - \text{黒の値}}$$

実際の1回目の赤成分では、黒9.8、白186.4、読取値137.5だったので、

$$\frac{137.5 - 9.8}{186.4 - 9.8} \approx 0.7231$$

となる。ログのNN入力R＝0.7231はこの値である。黒の基準は0、白の基準は1になる。範囲外の値を0〜1へ収める処理がクリップである。

Denseの計算は「掛けて足す」の繰返し

構造は「RGBの3入力→H個の計算→H個の計算→RGBの3出力」である。途中の2段が隠れ層で、H40なら各段に40個の計算単位がある。

1つの計算単位がすることは、$z = w_R R + w_G G + w_B B + b$。入力に掛ける係数wが重み、最後に足す調整値bがバイアスである。

実際のコードから、最初の段の1計算単位分を抜き出すと次のようになる。rは計算単位の番号、cはRGBの番号である。

```
float value=m.b1[r];
for(int c=0;c<3;c++) value+=m.w1[r*3+c]*input[c];
layer1[r]=value>0.0f?value:0.0f;
```

バイアスから合計を始め、RGBそれぞれに重みを掛けて足す。最後の行は「正ならそのまま、負なら0」にするReLUである。r*3+cは、並べて保存した重みの使う場所を指定する。

レポートの式 $h_1 = \mathrm{ReLU}(W_1 x + b_1)$ は、この処理を全計算単位についてまとめて書いたもの。入力xから途中の結果hを作り、次の段へ渡す。最後に得た3出力を0〜1へ収め、255倍して四捨五入すると、PPMの整数RGBになる。

3．学習の式とL1正則化は何をしているか

学習は、予測した色を正解へ近づけるように重みを更新する作業である。PCで、135行を使い、V9の重みから40,000回の更新を行った。3×3の正解画像にはresult.ppmを使用した。

学習式の意味は「色の誤差＋重みを大きくしすぎない罰則」である。 実際の式は次のとおり。

$$\mathcal{L} = \frac{1}{\sum_i a_i} \sum_i \frac{a_i}{3} \sum_{c\in\{R,G,B\}} \ell_\beta(\hat{y}_{i,c} - t_{i,c}) + 10^{-6} \sum_{\ell=1}^{3} \sum_{j,k} |(W_\ell)_{jk}|.$$

記号	意味
i、c	学習行の番号とRGBの成分
$\hat{y}$、t	予測値と正解。学習ではRGBを255で割った尺度を使う
a_i	学習行ごとの重要度。ネットワーク内の重みWとは別
$\sum$	指定範囲の値を全て足す記号
ℓ、j、k	層の番号と、その重みの表の中の位置

前半は色の誤差を、各行の重要度に従って平均する部分。誤差にはSmooth L1を使った。

$$\ell_\beta(e) = \begin{cases} e^2/(2\beta) & (|e| < \beta), \\ |e| - \beta/2 & (|e| \geq \beta), \end{cases} \quad \beta = 0.05$$

eは予測と正解の差。小さい誤差では二乗、大きい誤差では直線的に増える。一部の大きな誤差だけに学習が強く引っ張られることを抑えるために使った。

後半がL1正則化で、全3層の重みの絶対値を足し、10^{-6}倍して加える。外側の$\sum$で層、内側の$\sum$で各重みを足す。重みが大きいほど罰則が増えるため、小さい重みを作る方向にも更新される。バイアスにはこの罰則を付けない。

Pythonでは、`loss_rows`でRGB誤差を求め、`(loss_rows*weights).sum()/weights.sum()`で重要度付き平均にし、L1項を加えた。`loss.backward()`で更新方向を計算し、`optimizer.step()`で重みを更新した。

beta＝0.05は学習式の切替値で、枝刈りの閾値とは別。学習時の損失も、結果表のMSEとは別の値である。

4．枝刈りとCSRは、それぞれ何をしたか

枝刈り＝小さい重みを0にする

$$|w| < \tau \quad \Rightarrow \quad w = 0$$

閾値τは重みを残す境目。H60では約0.0543だった。例えば0.03や−0.03は0になり、0.2や−0.2は残る。符号ではなく絶対値で判断し、バイアスは変えない。

候補は学習済み重みの値から作った。別の8×8チャートの128入力で、MSE増加5%以下かつ容量が小さくなるものを残し、学習内9入力でMSE300以下も満たす候補から最小容量を選んだ。新規10回の測定中は固定した。

CSR＝0を省き、残した値と位置を保存する

説明用の小さな例を考える。これは実測の重みではない。

```
元の表        [ 0,    0.5, 0   ]
              [-0.2,  0,   0.1 ]
values  = [0.5, -0.2, 0.1]   ← 0以外の値
columns = [1,    0,   2  ]   ← 元の列番号（0始まり）
rows    = [0,    1,   3  ]   ← 各行の開始位置と終端
```

rowsの0〜1の手前が1行目、1〜3の手前が2行目なので、1行目はvaluesの0番だけ、2行目は1番と2番を使う。入力(x_0, x_1, x_2)に対し、$0.5x_1$と$-0.2x_0 + 0.1x_2$を計算できる。

実際のArduino関数は次のとおり。layer.colがcolumns、layer.rowがrowsである。

```
void sparseLayer(const comparison_models::SparseLayer &layer,const float *input,float
↪  *output,bool relu) {
  for(int r=0;r<layer.rows;r++) {
    float value=layer.bias[r];
    for(uint16_t k=layer.row[r];k<layer.row[r+1];k++) value+=layer.values[k]*input[layer.col[k]];
    output[r]=(relu && value<0)?0:value;
  }
}
```

rは行、kは保存した重みの番号。バイアスを用意し、その行に残った重みだけを対応する入力へ掛けて足す。ReLUは隠れ層だけで使う。

CSR変換の正しさは、枝刈り後の重みを通常配列で計算した結果と照合した。**圧縮の精度への影響**は、枝刈り前のDenseと比較した。変換が正しくても、枝刈りで色の誤差は増える場合がある。

5．MSEと「10回平均」「10回目」の違い

MSEは、補正後の色yと先生指定画像の正解tのずれ。9色×RGBの3成分＝27個を比べる。

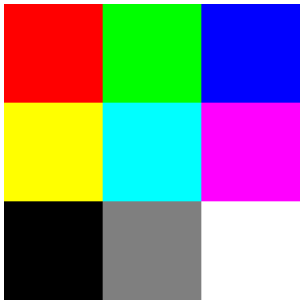
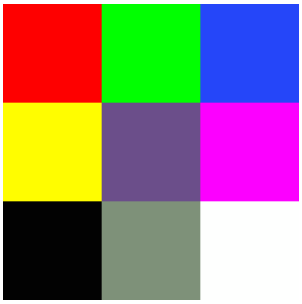
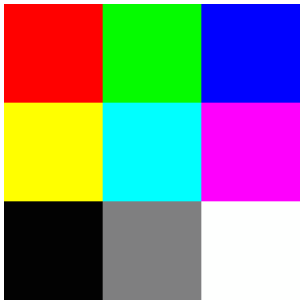
$$\mathrm{MSE} = \frac{1}{27}\sum_{i=1}^{9}\sum_{c\in\{R,G,B\}}(y_{i,c}-t_{i,c})^2$$

27個それぞれの差を二乗して足し、27で割る。説明用に、全ての差が10ならMSEは100になる。0なら全て一致し、小さいほど正確である。

レポートの結果表は、次の10回平均を使っている。 MSEは小数第2位まで表示し、第3位以降を切り捨てる。平均は切り捨て前の値で計算する。

H	Dense平均MSE	枝刈り＋CSR平均MSE
40	319.56	452.04
60	238.33	493.51
80	231.68	465.72
100	260.42	651.92

例えばH60のDenseは、10回目の画像のMSEが2.33だが、10回平均は238.33である。

先生指定の正解	H60 Dense・2回目	H60 Dense・10回目
		
比較の基準	MSE 2339.37	MSE 2.33

2回目の大きな誤差も含めた全10回の平均なので、平均は高くなる。操作ミスの申告がなかったため、その回も含めた。平均値に対応する実測画像が1枚あるわけではなく、10回目の画像へ平均MSEを付けて、その1枚の誤差として扱うことはできない。

平均は $\overline{m} = (m_1 + \cdots + m_{10})/10$。SD（標本標準偏差）は各回がどれだけばらついたかを表す。平均が小さくても、SDや最大値が大きければ、毎回安定して正確とはいえない。

6．容量の式と、Flash・RAMの違い

DenseとCSRの容量を手計算する

FP32は数値1個に4 byte使う。Denseの重み数は $3H + H^2 + 3H$、バイアス数は $H + H + 3$ なので、

$$B_{\text{Dense}} = 4(H^2 + 8H + 3)$$

H60なら重み3,960個、バイアス123個で、$4(3,960 + 123) = 16,332$ byteになる。

CSRでは、残った重みK個が4K byte、列番号がK byte、行の開始位置が $2(2H + 6)$ byte、バイアスが $4(2H + 3)$ byteになる。

$$B_{\text{CSR}} = 4K + K + 2(2H + 6) + 4(2H + 3) = 5K + 12H + 24$$

H60はK＝1,141なので、$4,564 + 1,141 + 252 + 492 = 6,449$ byteである。削減率は $100(1 - 6,449/16,332) = 60.51\%$。位置情報にも容量が必要なので、重みがあまり減らない場合はCSRでも小さくならない。

モデルだけの容量と、プログラム全体を分ける

Flashはプログラム等を保存する領域、RAMは動作中の作業領域である。1モデルずつ入れたプログラム全体のFlashは次のようになった。

H	Dense Flash byte	CSR Flash byte	削減率
40	60,852	56,020	7.94%
60	69,492	59,612	14.22%
80	81,348	62,396	23.30%
100	96,388	66,260	31.26%

H60のモデル配列は60.51%減ったが、Flash全体は14.22%の削減だった。Arduinoの基本処理など、両方式で共通の部分が残るためである。

静的RAMは全8条件で6,764 byteと同じ。32,768−6,764＝26,004 byteは静的割当後の残りであり、ローカル変数が実際に使った量ではない。

関数内で一時的に使う領域がスタック。元の中間配列はfloat[100]を2本で、宣言上800 byteだが、他の処理もあるため最大使用量は別に測る必要がある。今回は未測定である。

7．速度のコードと、測った時間の範囲

使ったのは、保存済みの実測9入力をArduinoへ繰り返し入れる方法。新たに紙を読む時間ではなく、色の補正を計算する区間を測った。

実際の関数は次のとおり。micros()は動作開始からの時間をμsで読み、1 μsは100万分の1秒である。

```
uint32_t timeModel(int model,int group,double &checksum) {
  float output[3];checksum=0;
  uint32_t begin=static_cast<uint32_t>(micros());
  for(int call=0;call<100;call++) {
    int input=(group*100+call)%comparison_models::BENCHMARK_INPUT_COUNT;
    comparison_models::predict(model,comparison_models::BENCHMARK_INPUTS[input],output);
    checksum+=output[0]+output[1]+output[2];
  }
  return static_cast<uint32_t>(micros())-begin;
}
```

コード	していること
begin=micros()	開始時刻を保存する
call<100	100回まとめて計算する
... % BENCHMARK_INPUT_COUNT	保存した9入力を順に繰り返す
predict(...)	指定モデルで色を補正する
checksum+=...	出力を足し、計算結果が使われるようにする
micros()-begin	100推論分の経過時間を返す

その時間を100で割って1推論当たりの値にした。各モデルは最初に10回動かしてから、100推論を5組計った。Dense先行とCSR先行を交互にした。

H	Dense μs/推論	CSR μs/推論	短縮率
40	465.262	305.834	34.27%
60	945.648	671.674	28.97%
80	1593.368	956.212	39.99%
100	2408.160	1348.392	44.01%

H60の945.648 μsは約0.000946秒、671.674 μsは約0.000672秒。計算時間は28.97%短くなった。

紙を動かす時間、センサ読取、シリアル出力、MSE計算は計時区間へ含めていない。一方、入力選択と出力の合計は含まれる。したがって、紙の読取全体や推論関数だけを単独で測った時間とは区別する。

8．結果をどう説明すればよいか

H60の1組で、指標をつなげて考える

指標	Dense	枝刈り＋CSR
新規10回の平均MSE	238.33	493.51
モデル配列容量 byte	16,332	6,449
プログラム全体のFlash byte	69,492	59,612
静的RAM byte	6,764	6,764
計算時間 µs/推論	945.648	671.674

小さくなり、速くなった。しかし、色の誤差は増えた。 全4種類のHでも、圧縮後の平均MSEは増えた。MSE増加5%以下と、圧縮方式の10回平均MSE300以下という目標は達成しなかった。

自分の言葉で説明する例

「色補正ニューラルネットワークを、通常のDenseと枝刈り＋CSRで比較しました。小さい重みを0にし、残った値と位置だけを保存して計算を減らしました。40・60・80・100次元で、同じ9色の入力を両方式に使って10回測りました。モデルの容量と計算時間は減りましたが、圧縮後のMSEが増えたため、精度維持は達成できませんでした。」

混同しやすい点

PASSの意味。 固定入力でPCとArduinoの計算が一致し、CSRの位置情報が正しいことを確認した。色が正解に近いかは別にMSEで判断するので、PASSでもMSEは悪くなり得る。

測定した範囲。 新規入力は学習や閾値選択に加えていないが、紙の色は学習にも使ったもの。学習に使っていない別の紙での最終確認は未実施である。

まだ測っていないもの。 H50・H200、各方式が何次元まで載るか、実行時スタックの最大量は未測定。今後は削る量の見直し、枝刈り後の追加学習、入力変動の確認、別の紙での評価が候補になる。

提出用の本文は report.pdf、色画像は color_images。このガイドは、その本文・保存済みコード・実測CSVの内容を説明したもので、測定値やモデルを変更していない。